

Git Commands Reference Guide

Snowflake Hotel Booking Project | August 2026

1. Initial Configuration (One Time Only)

Before using Git for the first time, tell Git who you are:

Configure your name and email:

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

💡 Why: Every commit records WHO made the change. Use same email as your GitHub account.

2. Creating Project Folder

Navigate to Desktop:

```
cd ~/Desktop
```

💡 Why: `cd` = change directory. `~` = your home folder. Moving to Desktop first.

Create project folder:

```
mkdir snowflake-hotel-booking-project
```

💡 Why: `mkdir` = make directory. Creates a new folder. Use hyphens (-) for GitHub repo names!

Go into project folder:

```
cd snowflake-hotel-booking-project
```

💡 Why: Move inside the folder so all future commands run here.

Confirm you are in right place:

```
pwd
```

💡 Why: `pwd` = print working directory. Shows exactly where you are in your computer.

3. Setting Up Folder Structure

Create folders for each layer:

```
mkdir setup
mkdir bronze
mkdir silver
mkdir gold
mkdir automation
```

💡 Why: Separate folders for each layer. Numbers in filenames (01_, 02_) show order to run them.

Create SQL files inside folders:

```
touch setup/01_setup.sql
touch bronze/02_bronze.sql
touch silver/03_silver.sql
touch gold/04_gold.sql
touch README.md
```

💡 Why: `touch` creates empty files. Git tracks FILES not folders — empty folders are ignored!

Verify folder structure:

```
ls -R
```

💡 Why: `ls` = list. `-R` = recursive. Shows all files and folders inside your project.

4. Initializing Git

Initialize Git repository:

```
git init
```

💡 Why: Tells Git to start tracking this folder. Creates hidden .git folder inside. Like installing CCTV in your project!

5. Checking Status

Check what Git sees:

```
git status
```

💡 Why: Shows which files are new, changed, or ready to save. Run this often!

⚠️ Note: Red files = untracked (Git sees but not saving yet)

⚠️ Note: Green files = staged (ready to commit)

6. Staging and Committing

Stage all files:

```
git add .
```

💡 Why: . = everything in current folder. Like putting files in envelope before sealing.

Stage specific file:

```
git add filename.sql
```

💡 Why: Only stage one specific file instead of everything.

Commit your code:

```
git commit -m "Your message here"
```

💡 Why: Saves snapshot of your code with description. -m = message. Write clear messages — future you will thank you!

7. Connecting to GitHub

Add GitHub as remote:

```
git remote add origin https://github.com/USERNAME/repo-name.git
```

💡 Why: Tells local Git WHERE to push code on internet. origin = nickname for your GitHub URL.

Push code to GitHub:

```
git push -u origin master
```

💡 Why: Uploads your code to GitHub! -u sets up tracking so next time just type git push.

⚠️ Note: GitHub requires Personal Access Token (ghp_...) as password — NOT your regular password!

8. Important Notes

- Git does NOT track empty folders — add a file first!
- GitHub no longer accepts regular passwords — use Personal Access Token (ghp_...)
- Use hyphens (-) for GitHub repo names, underscores (_) for SQL object names
- Always run git status before and after staging to confirm changes
- Write clear commit messages — future you will thank you!
- automation/ folder will not show on GitHub until you add files inside it